SkillForge

AI System for Generating, Packaging, Evaluating, and Optimizing Agent Skills

Project Specification & Technical Architecture

SkillForge is a website and AI pipeline that creates **skills** for AI agents. A user names any skill they need, technical or non-technical (for example AI Engineer, Marketing Strategist, Contract Reviewer, Flutter Developer). If no good skill exists, SkillForge researches the domain, designs a validated skill specification and generates an installable Markdown skill package (`SKILL.md` plus reference files). The package drops into an agent such as Claude Code or Codex, and the agent then **acts as that skill**, the same way these agents already load skills and plugins from their `.claude/` and `.agents/` folders.

Core idea: natural-language skill request → domain knowledge collection → canonical SkillSpec → platform compiler → installable skill package (.md) → evaluation → optimization.

Document	Project Specification v2.0. Replaces the AgentForge specification v1
Project name	SkillForge (formerly misspelled "AgentForge")
Date	22 October 2026
AI Lead	Ziad Abdullah: LLM, web scraping, NLP
Status	Draft for team review

1. Project Vision

General-purpose agents such as Claude Code and Codex are strong generalists, but they do their best work when they are given a **skill**: a focused package of expertise, procedures, templates and guardrails that the agent loads when a task needs it. Today a few skills are written by hand, mostly for popular programming topics. For most roles and domains, such as AI engineering, marketing, finance, HR, legal review or teaching, no good skill exists. Writing one well is hard: it needs domain knowledge, knowledge of each platform's skill format, and testing to confirm the skill actually triggers and improves results.

SkillForge closes this gap. It goes beyond asking an LLM to "write a SKILL.md". It reasons about the requested role, collects and verifies domain knowledge from the web, designs the skill's structure, produces a canonical specification, compiles it for each agent platform, and evaluates whether the generated skill really improves the agent.

1.1 Problem statement

- Agents behave generically unless they are given role-specific knowledge and procedures.
- Existing skills cover a narrow set of mostly technical topics. Non-technical roles are almost uncovered.
- Hand-written skills are inconsistent in quality, rarely tested, and tied to one platform's format.
- No one measures whether a skill actually helps, or whether it activates at the right time.

1.2 Objectives

#	Objective	Success measure (target for V1 unless stated)
O1	Generate a complete, installable skill package for **any** requested role or task, technical or non-technical.	≥ 90% of requests produce a package that passes validation without manual edits.
O2	Ground every skill in real domain knowledge collected from the web, with citations.	Every factual reference file lists its sources. No uncited claims in reference files.
O3	Compile one platform-neutral SkillSpec into multiple agent platforms.	Claude Code and Codex packages generated from the same spec. ChatGPT adapter in V2.
O4	Prove that generated skills improve agent output.	Measurable quality gain over the no-skill baseline on each skill's benchmark (V2).
O5	Improve skills automatically from evaluation results.	Optimized version beats the first generated version on the same benchmark (V3).
O6	Make it usable by non-technical people through a website.	A user goes from request to downloaded skill in under 10 minutes.

1.3 Scope

In scope	Out of scope (for now)
Website for describing, reviewing, previewing and downloading skills	Building or hosting the agents themselves. SkillForge produces skills, the user's agent runs them
Role skills (e.g. AI Engineer) and task skills (e.g. Write a press release)	Fine-tuning or training models
Web knowledge collection with citations; optional repository analysis for codebase-specific skills	Executable tool servers (MCP servers). Skills may reference tools but SkillForge does not build them in V1
Compilers for Claude Code and Codex (V1), ChatGPT and generic Markdown (V2), others later	Paid marketplace and billing (considered after V3)
Evaluation and automatic optimization of skills	

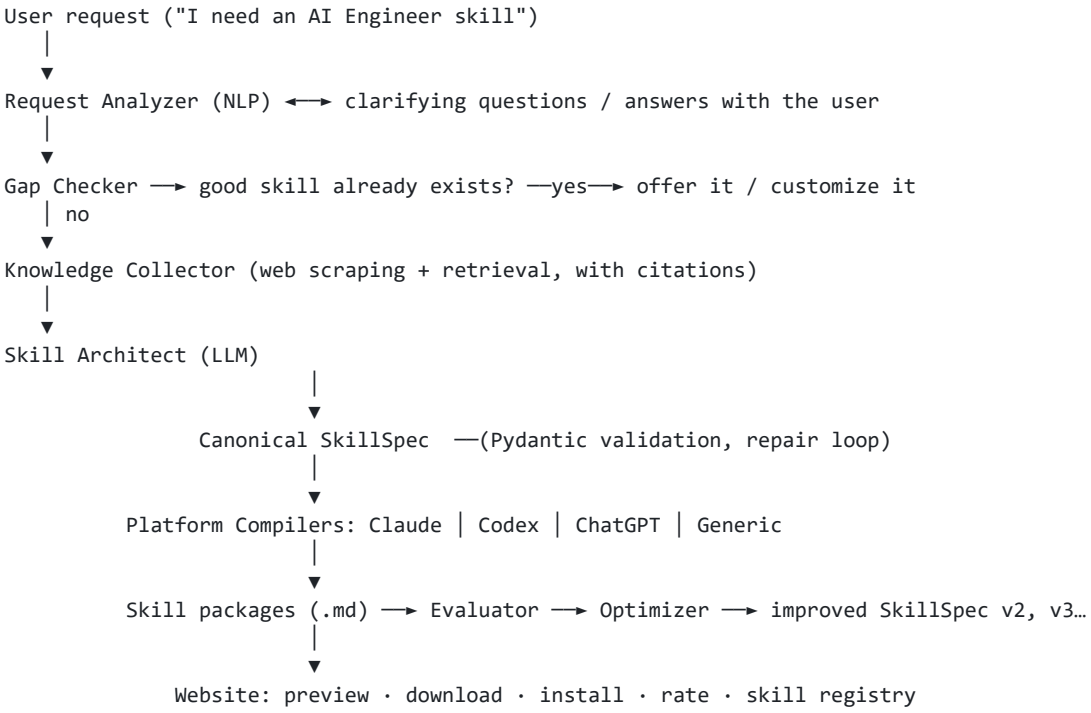
1.4 Target platforms

Initial targets: **Claude Code** and **Codex**. Both load skills as folders containing a `SKILL.md` file. Next: **ChatGPT** and a **generic Markdown** export for any agent that reads instruction files. Future adapters: Gemini CLI, Cursor, Windsurf, Aider and others.

2. Key Concepts

Term	Meaning in SkillForge
Skill	A folder that teaches an agent one capability or role. Its entry point is `SKILL.md`: a short YAML header (`name`, `description`) plus Markdown instructions. It can include reference files, templates and scripts.
Role skill	Makes the agent act as a professional, e.g. ai-engineer, marketing-strategist. Contains principles, procedures for that role's typical tasks, output templates and guardrails.
Task skill	Teaches one repeatable job, e.g. write-press-release, review-nda.
Trigger	The `description` field tells the agent when to load the skill. A skill with a poor description is never used, so trigger quality is measured explicitly (§8).
Progressive disclosure	The agent always sees only the skill's name and description. It reads the `SKILL.md` body when the skill triggers, and opens reference files only when needed. SkillForge keeps `SKILL.md` short and puts deep knowledge into `references/`.
Skill package	The generated output: the skill folder for one platform, ready to copy into `.claude/skills/`, `.agents/skills/`, or a plugin.
SkillSpec	SkillForge's platform-neutral intermediate representation (IR) of a skill: the single source of truth from which every platform package is compiled.

3. System Architecture



Major components

1. **Request Analyzer**: turns a free-text request into structured fields: skill kind (role / task), domain, audience, expected outputs, risk level, target platforms. Generates clarifying questions when the request is vague.
2. **Gap Checker**: searches the SkillForge registry and public skill collections for an existing skill. If a good one exists the user can use or customize it. Generation runs only for skills that do not exist.
3. **Knowledge Collector**: gathers domain knowledge from the web (best practices, official guides, standards, checklists, templates), cleans and deduplicates it, and keeps citations. For codebase-specific technical skills it can also analyze a repository.
4. **Skill Architect**: designs the skill from the analyzed request and the collected knowledge: triggers, principles, procedures, outputs, guardrails, references and evaluation prompts. It writes a SkillSpec, never final Markdown.
5. **SkillSpec**: platform-independent, validated intermediate representation and source of truth.
6. **Platform Compilers**: translate a SkillSpec into each platform's package format with templates and deterministic validation.
7. **Evaluation Engine**: tests whether the skill triggers correctly and improves the agent's output.
8. **Optimization Engine**: analyzes evaluation failures, edits the SkillSpec and keeps the best version.
9. **Website and Registry**: user interface, skill library, versions, ratings and downloads.

4. Canonical SkillSpec

The LLM must not generate the final Markdown directly. It fills a validated intermediate representation. Pydantic validates every SkillSpec before compilation, and validation errors go back to the Skill Architect so it can repair its output. The SkillSpec reuses the design and validation code of the AgentSpec v0 (task AF-1): strict models, closed vocabularies, stable ids and cross-field rules.

Example: a role skill that did not exist before.

```yaml
spec_version: "0.2"
skill:
  name: ai-engineer                # kebab-case, becomes the folder name
  version: "1.0"
  title: AI Engineer
  kind: role                       # role | task | workflow
  domain: technology/ai
  summary: Senior AI engineer who designs, builds and evaluates LLM applications.
triggers:
  use_when:
    - designing, building or debugging LLM, RAG or agent systems
    - choosing models, embeddings, prompts or evaluation methods
  avoid_when:
    - frontend or backend work with no AI component
expertise:
  principles:
    - Define the evaluation metric before choosing a model
    - Prefer retrieval and prompting over fine-tuning until data proves otherwise
procedures:
  - id: design-rag
    title: Design a RAG system
    steps: [clarify use case and data, choose chunking, choose retriever,
            define evaluation set, propose design with trade-offs]
outputs:
  - id: design-doc
    template: templates/design-doc.md
guardrails:
  - id: no-fabricated-numbers
    rule: Never invent benchmark results or prices; cite a source or say unknown.
    severity: must
tools:                             # abstract capabilities, never vendor names
  optional: [filesystem_read, terminal, web_search]
knowledge:
  references:
    - id: rag-patterns
      file: references/rag-patterns.md
      sources: [src-1, src-2]
sources:
  - id: src-1
    url: <scraped page URL>
    retrieved: 2026-10-22
evaluation:
  test_prompts:
    - Design a question-answering system over 10,000 internal PDFs
  rubric: [asks clarifying questions, defines eval metric, states trade-offs]
targets: [claude, codex]
```

Validation rules (examples)

- Unknown fields are rejected, so typos never pass silently.
- `name` is kebab-case and unique in the registry. The compiled description stays within each platform's length limit.
- Every reference file lists at least one source. Every source id that is used must exist.
- Every procedure step and output template referenced must be defined.
- A role skill needs at least one procedure and one guardrail. High-risk domains (legal, medical, finance) require a disclaimer guardrail and human-review guidance.
- Tools are abstract capabilities (`terminal`, `web_search`), never platform-specific tool names.

5. Knowledge & Domain Intelligence

Non-technical skills have no repository to analyze, so their knowledge comes from the web. The Knowledge Collector uses deterministic tools first and uses the LLM only for semantic work: summarizing, structuring and checking claims against sources.

Capability	Recommended tool	Purpose
Source discovery	Web search API	Find authoritative guides, standards, official docs, course material for the domain
Static page fetching	httpx + robots.txt check	Polite, rate-limited fetching that respects site rules
Dynamic pages	Playwright	Pages that need JavaScript to render
Main-content extraction	trafilatura / BeautifulSoup	Strip menus and ads, keep article text and structure
Language processing	spaCy	Key phrases, entities, sentence splitting, language detection
Deduplication	MinHash / embeddings	Remove repeated and near-duplicate content
Retrieval	Embeddings + pgvector	Give the Skill Architect the most relevant passages, with their sources
Existing-skill corpus	Public SKILL.md collections (license-checked)	Gap checking, style examples, benchmark data
Codebase analysis (optional)	Tree-sitter, ripgrep, Git	Technical skills tied to one repository
Semantic interpretation	LLM	Summaries, procedures and checklists built from retrieved sources

> **Responsible scraping:** respect robots.txt and site terms, rate-limit every domain, store only what is needed, keep the URL and retrieval date of every source, and do not copy long copyrighted passages into skills. Skills contain summarized, cited knowledge, not copied pages.

6. Platform Compiler Architecture

```
SkillSpec
   ↓  platform rules (folder layout, header fields, length limits)
   ↓  templates (Jinja2): SKILL.md, references/, templates/
   ↓  LLM refinement (wording and clarity only, never structure)
   ↓  package validator (header valid, links resolve, sizes within limits)
Skill package (.md files)  →  zip download / install into the agent's folder
```

Templates and deterministic validation control the final format. The LLM improves wording and clarity but does not invent structure.

Generated package (Claude Code example)

```
.claude/skills/ai-engineer/
├── SKILL.md                    ← header + short instructions (always the entry point)
├── references/
│   ├── rag-patterns.md         ← deep knowledge, opened only when needed
│   └── evaluation-checklist.md
└── templates/
    └── design-doc.md           ← output template the agent fills in
```

```
--- SKILL.md ---
---
name: ai-engineer
description: Acts as a senior AI engineer. Use when designing, building, evaluating
  or debugging LLM, RAG or agent systems. Not for general web development.
---
# AI Engineer
## How to work
1. Clarify the use case, the data and the success metric before proposing a design.
2. ...
## References
- Retrieval design: read references/rag-patterns.md
```

Platform targets

Platform	Where the skill is installed	Package	Phase
Claude Code	`.claude/skills/<name>/` in a project, the user-level skills folder, or inside a plugin	SKILL.md + references/templates; optional plugin bundle	V1
Codex	`.agents/skills/<name>/`	SKILL.md + references/templates	V1
ChatGPT	To be confirmed when the adapter is designed. Current skill / custom-GPT support must be verified	SKILL.md where supported, otherwise instructions + knowledge files	V2
Generic	Any agent that reads Markdown instruction files (e.g. an AGENTS.md section)	Single Markdown file	V2
Gemini CLI, Cursor, others	Per platform	Per platform	Later

Suggested structure:

```
platforms/
├── base.py          ← shared compiler interface
├── claude/          ← compiler + templates
├── codex/
├── chatgpt/
└── generic/
```

7. Website & User Flow

1. **Describe**: the user types what they need ("a marketing skill for B2B SaaS launches") and picks target platforms.
2. **Clarify**: SkillForge asks 3–5 short questions (audience, tone, typical tasks, things to avoid).
3. **Check**: if a matching skill exists in the registry, the user can use it, customize it or generate a new one.
4. **Generate**: knowledge collection and design run with visible progress. The user sees the sources being used.

5. **Review**: the user reviews the skill outline (procedures, guardrails, references) and can edit it before compiling.
6. **Preview**: side-by-side preview of the generated files for each platform.
7. **Install**: download a zip, or copy the folder path and install command for Claude Code or Codex.
8. **Rate**: after use, the user rates the skill. Ratings feed the optimizer and the registry ranking.

Main pages: Home · Create skill · Skill library (search, categories, ratings) · Skill detail (versions, sources, files) · My skills · Evaluation report.

8. Evaluation Framework

Evaluation is the key differentiator. Every generated skill gets a small benchmark: test prompts produced with the skill and refined by humans. The agent runs each prompt **with** and **without** the skill, and the difference is measured.

Metric	Purpose	How it is measured
Trigger accuracy	Skill activates when it should and stays silent when it should not	Positive and negative prompts; precision / recall of activation
Task quality uplift	Skill makes the agent's output better	LLM-as-judge with a rubric, with vs without the skill; human spot checks
Procedure adherence	Agent follows the skill's steps and templates	Rubric items per procedure
Guardrail violations	Unsafe, prohibited or out-of-scope behavior	Count of broken must-rules
Factual grounding	Claims match the cited sources	Claim-to-source checking on reference files
Cross-platform consistency	Same spec behaves similarly on Claude and Codex	Score difference between platforms
Token cost	Efficiency of the skill's size and loading	Tokens loaded per task
Execution time	Operational efficiency	Time per task
User rating	Real-world usefulness	Ratings and feedback on the website

9. Automatic Skill Optimization

```
Skill request ↓ Generate SkillSpec v1 ↓ Compile ↓ Run benchmark ↓ Evaluate
   ↓ Analyze failures ↓ Modify SkillSpec ↓ Generate v2 / v3 / … ↓ Keep the best version
```

The optimizer changes SkillSpec fields, never the compiled Markdown, so every variant stays valid and comparable. Optimization variables:

Variable	SkillSpec field(s) changed
Trigger wording	`skill.summary`, `triggers.use_when / avoid_when`
Procedure detail	`procedures[].steps`
Knowledge depth and placement	`knowledge.references` (what goes in SKILL.md vs references/)
Guardrail strength	`guardrails[].severity` and rule wording
Output structure	`outputs[].template`
Examples	Number and kind of worked examples
Granularity	One role skill vs several smaller task skills

10. Recommended Technology Stack

Layer	Technology
Language	Python 3.11+
API	FastAPI
Schema / validation	Pydantic
LLM abstraction	LiteLLM
AI workflows	LangGraph
Web scraping	httpx, Playwright, trafilatura, BeautifulSoup
NLP	spaCy, sentence embeddings
Templates	Jinja2
Database	PostgreSQL
Vector search	pgvector
Job queue (long generation runs)	Celery or arq with Redis
Frontend	Next.js + TypeScript
UI	Tailwind CSS + shadcn/ui
Containers	Docker
CI/CD	GitHub Actions
Observability	OpenTelemetry
Experiment tracking	MLflow or Weights & Biases

11. Recommended Repository Structure

```
skillforge/
├── apps/
│   ├── api/                  ← FastAPI backend
│   └── web/                  ← Next.js website
├── core/
│   ├── spec/
│   │   ├── models.py         ← SkillSpec (evolves from AgentSpec v0)
│   │   ├── validator.py
│   │   └── schema.json
│   ├── analysis/
│   │   ├── request.py        ← Request Analyzer (NLP)
│   │   └── gap_checker.py
│   ├── knowledge/
│   │   ├── search.py
│   │   ├── scraper.py
│   │   ├── extract.py
│   │   └── retrieval.py
│   ├── repository/           ← optional codebase analysis
│   ├── agents/
│   │   ├── architect.py      ← Skill Architect
│   │   ├── evaluator.py
│   │   └── optimizer.py
│   └── compiler/
│       ├── base.py
│       ├── claude.py
│       ├── codex.py
│       ├── chatgpt.py
│       └── generic.py
├── templates/  (claude/ codex/ chatgpt/ generic/)
├── evaluation/ (benchmarks/ metrics/ runner.py)
├── examples/skillspec/       ← ai-engineer.yaml, marketing-strategist.yaml, …
├── tests/
├── docs/
└── pyproject.toml
```

12. API & CLI Design

The website uses the API. The CLI exposes the same pipeline for development and research.

API endpoint	Purpose
POST /skills/requests	Submit a skill request, returns clarifying questions
POST /skills/requests/{id}/answers	Answer questions, start the generation job
GET /skills/jobs/{id}	Progress, sources found, draft SkillSpec
POST /skills/{id}/compile?platform=claude	Compile a package for one platform
GET /skills/{id}/download?platform=codex	Download the package as a zip
GET /skills?query=…	Search the skill registry
POST /skills/{id}/evaluate	Run the skill's benchmark

```
skillforge create "AI Engineer"                 # analyze → collect → design → validate
skillforge create --from-file request.md
skillforge compile ai-engineer --platform claude
skillforge compile ai-engineer --all
skillforge evaluate ai-engineer
skillforge optimize ai-engineer --rounds 3
skillforge install ai-engineer --platform codex --path ./my-project
```

13. Team Roles

Role	Owns
AI Lead (LLM, web scraping, NLP): Ziad Abdullah	SkillSpec schema · Request Analyzer · Gap Checker · Knowledge Collector (scraping, extraction, retrieval) · Skill Architect and repair loop · LLM-as-judge evaluation · Optimizer
Backend	FastAPI services, job queue, database, registry, authentication, storage of packages
Platform compilers	Compiler interface, Claude/Codex/ChatGPT/generic compilers, templates, package validators
Frontend	Website: create flow, previews, library, evaluation reports
Evaluation / research	Benchmark design, human evaluation, experiment tracking (shared with AI Lead)

14. Development Roadmap

V1: Minimum Viable Product

Skill request → request analysis → web knowledge collection → SkillSpec → Claude and Codex compilers → downloadable skill package. Basic website (create, preview, download). Ten seed skills across technical and non-technical domains.

V2: Skill Evaluation

Per-skill benchmarks, execution harness on Claude and Codex, trigger-accuracy and quality-uplift scoring, regression detection, evaluation reports on the website. ChatGPT and generic Markdown adapters. Skill registry with search and ratings.

V3: Skill Evolution

Automatic generation of skill variants, benchmarking, failure analysis, SkillSpec modification and retention of the best version. Learning from user ratings. Skill bundles (several skills plus a plugin manifest for one role or team).

Immediate next steps

1. Evolve AgentSpec v0 (AF-1) into SkillSpec v0.2. Keep the validation design, add skill fields (§4). Freeze the schema at the planned v1 milestone (AF-6, 5 November 2026).
2. Write 3 example SkillSpecs (one technical, two non-technical) with tests, as done for AF-1.
3. Build the Claude compiler first and install a generated skill in a real project to confirm it triggers.
4. Prototype the Knowledge Collector on one non-technical domain (e.g. marketing).

15. Research Direction

Research question: Can LLMs automatically design, ground and optimize reusable agent skills for arbitrary professional domains, technical and non-technical, from a short natural-language request?

Experimental comparison

1. No skill (baseline agent)
2. Manually written skill
3. One-shot LLM-generated SKILL.md
4. SkillForge-generated skill (grounded, validated)
5. SkillForge-optimized skill

All five are compared on the same benchmark tasks per domain, reporting trigger accuracy, task-quality uplift, guardrail violations, factual grounding, cross-platform consistency, token cost and execution time. Domains are split between technical (e.g. AI engineering, Flutter) and non-technical (e.g. marketing, HR, legal review) to test generality.

16. Risks & Mitigations

Risk	Mitigation
Hallucinated domain knowledge	Citations required by validation; claim-to-source checks; human review for high-risk domains
Scraping legal / ethical issues	robots.txt and terms respected, rate limits, summaries not copies, source log
Platform skill formats change	Format rules isolated in each compiler; package validators updated per platform release
Skills that never trigger	Trigger accuracy is a first-class metric; the optimizer tunes descriptions
Harmful skills (e.g. requests for malicious capabilities)	Request Analyzer screens requests; refusal policy; mandatory guardrails
High LLM cost per skill	Caching of collected knowledge per domain; smaller models for extraction; registry reuse

17. Key Architectural Principle

Keep SkillSpec independent of any model or vendor. Claude Code, Codex, ChatGPT, Gemini, Cursor and future platforms are compilation targets. SkillSpec is the intermediate representation (IR) for agent skills: many inputs (requests, web knowledge, repositories), one IR, many outputs.

This makes SkillForge extensible to new agents, testable at every stage, and suitable for research into automated skill design, instead of being tied to a single agent product.